

Knowledge Graphs and LLMs

Model your domain as a Property Graph, build it with an LLM, query it with Cypher

Course: KG and LLM Integration Instructor: Gianmaria Silvello

2 ECTS

Goal

Build a small property graph for a domain you already know or you are working on, populate it from a (short) text source assisted by a large language model, and interrogate it with Cypher. The intellectual core is modeling. The LLM is a tool for extraction and consistency checking (also querying can be handled with LLMs). The graph is the artifact that proves your model holds.

Choose one track

Both tracks produce the same deliverables and are graded with the same rubric. Pick the track that matches the tools you already have.

Track A — Browser-only	Track B — API + notebook
Audience: no Python required. LLM: any chat interface you already use (ChatGPT, Claude, Gemini, Copilot, Mistral chat). No API key needed. Graph store: Neo4j AuraDB Free or Neo4j Sandbox. Free, no install, runs in the browser. Load: paste Cypher <code>CREATE/MERGE</code> statements (or <code>LOAD CSV</code>) in Neo4j Browser. Queries: Cypher typed and run in Neo4j Browser.	Audience: comfortable with Jupyter at the level of notebook 09. LLM: any API you have access to (OpenAI, Anthropic, Gemini, Azure OpenAI, a local Ollama model). Graph store: pick one: (i) Neo4j AuraDB Free via the <code>neo4j</code> Python driver; or (ii) <code>kuzu</code> (https://kuzudb.github.io/) embedded LPG with Cypher, installed by <code>pip install kuzu</code>, no server. Load: generate Cypher programmatically from the LLM response and execute it from a notebook. Queries: Cypher executed from a Jupyter notebook.

On the LLM choice. Use whatever access you have. A free-tier chat interface is acceptable. Document your prompts and one raw response so a reader can reproduce your work. If you used multiple models, say which produced which output.

Scope

Domain. Pick a domain you understand well: your research field, a hobby, a small dataset you curate (a music catalogue, a legal corpus, lab notebooks, a literary character network, a small product catalogue, archaeological finds, a clinical case series). Avoid sensitive personal data.

Text source. One to three short documents totalling no more than ten pages: abstracts, encyclopedia entries, catalogue records, statute fragments, lab notes. The source must be the basis of your facts; do not let the LLM invent entities not present in the text.

Target size. Roughly 15–30 nodes and 25–50 relationships. Small enough to inspect by eye, large enough to test queries.

Effort. Approximately 20 hours of work, consistent with 2 ECTS.

Six stages

1. Domain and competency questions

Write five competency questions that your graph must answer. Competency questions are the specification: they drive the schema in Stage 2 and the queries in Stage 5. Examples: 'Which authors influenced X?', 'Which compounds were tested on Y between 2010 and 2020?', 'Which statutes reference article Z?'. Avoid yes/no questions; prefer questions that require traversing a relationship or aggregating.

2. Property-graph schema

Define node labels, relationship types, and the properties carried by each. Submit a small diagram (hand-drawn, draw.io, Mermaid, or equivalent) and a table listing every label and relationship type. At least one relationship must carry properties (a qualified relation), for example a TREATED_WITH relationship with properties dosage and start_date, or a CITES relationship with year and page. Justify two modeling choices in one paragraph each: one decision where you modeled something as a node rather than a property, and one decision where you introduced a qualified relationship rather than a plain edge.

3. LLM-assisted extraction

Use your LLM (chat or API) to extract facts from the text source. Constrain the output to your Stage-2 schema. Start with a generic prompt; if the output is noisy, refine it into a constrained prompt that lists allowed labels and relationship types and asks the model to emit Cypher (or a structured list of triples). Save every prompt you used and one sample raw LLM response. Treat the LLM output as a draft; you accept, reject, or correct each fact against the text.

4. Build the graph

Load the validated facts into your graph store. Track A pastes CREATE/MERGE statements (or a LOAD CSV call) into Neo4j Browser. Track B executes the same statements through the neo4j Python driver or through kuzu. Add at least one uniqueness or existence constraint on a key property (for example, a unique identifier on the main entity).

5. Cypher queries

Write one Cypher query per competency question. Include the query text, the result, and a one-line comment on whether the result matches what you expected from the source. At least one of your queries must include a multi-hop pattern and at least one must use aggregation (COUNT, COLLECT, or AVG).

6. Reflection and error analysis

In one and a half to two pages, discuss: where the LLM produced facts not supported by the text (hallucination), where it merged or split entities incorrectly (coreference and disambiguation), and where domain jargon or style variance reduced extraction quality. Name up to three failure modes with concrete examples from your own output. Briefly compare your initial prompt with the refined one.

Deliverables

- A report of max five pages (PDF) covering Stages 1–6.
- A Cypher script file with constraints, CREATE/MERGE statements (or LOAD CSV with the CSV attached), and the queries from Stage 5.
- A prompt log: every prompt used in Stage 3 and one example raw LLM response.
- Either a graph export (Cypher dump, JSON, or a .dump file) or a read-only link to your AuraDB instance.

Practical guidance

- Pick a tight scope. A graph that answers five well-posed questions on a small corpus scores higher than a sprawling graph that answers none cleanly.
- Keep the schema rich, not the data. Model qualified relationships, multiple node labels, and meaningful properties; do not pad the data.
- Cite the text. Every node and relationship you accept must be traceable to a sentence in the source.
- Iterate prompts in writing. Save the first prompt; save the refined prompt; explain in one line what you changed and why.
- Avoid sensitive personal data and copyrighted full texts. Public abstracts, public catalogues, and statutes are safe.
- If you use Neo4j Sandbox, your instance expires after a few days; export the graph (Cypher dump) before it disappears, or use AuraDB Free which is persistent.

Submission

Submit the report (PDF), the Cypher script, the prompt log, and the graph export or AuraDB link as a single ZIP archive. Name the archive surname_kglm_assignment.zip.

Submission deadline: When the work is ready but no later than **10 July 2026**.

No extension will be granted.